PyRetinotopy

PsychoPyによるレチノトピー刺激呈示システム

プロジェクト・実験ガイド / 日本語版

責任者・開発者	版・改訂日
番浩志	v0.4.0 / 2026-11-10

プロジェクトの目的

fMRIを用いて視野位置に対応する皮質表象を調べるため、レチノトピー刺激を柔軟に定義し、単眼・両眼の表示装置で時間精度に配慮して呈示することを目的としています。刺激はMEG・EEG・視覚心理物理実験にも応用できます。

原版から引き継ぐ特徴

番浩志が開発・公開してきたMATLAB Psychtoolbox 3版Retinotopy刺激セット（https://github.com/hiroshiban/Retinotopy）を、PythonとPsychoPyへ移植したプロジェクトです。多彩な刺激、物体画像とRDS、被験者別設定、立体視の表示方式、ラン開始からの累積時刻による制御を共通のプログラムで扱います。

刺激	運用
14基本モード × 5キャリア・課題 = 70モード	v0.4.0 / 2026-11-10
単画面・左右分割・上下分割・anaglyph・行列交互・2画面・quad buffer	NumPy画像バンクと取得前の全画像準備

本書の使い方

本書はプロジェクトの説明から、環境設定、実際の取得、時刻の点検、引用と謝辞までをまとめています。コマンドはWindows PowerShellを例にしています。装置の校正値と刺激条件は実験室の設定へ変更してください。

PyRetinotopyの開発には、OpenAIの生成AI（Codexを含む）のサポートを利用しています。

PyRetinotopyの作者・責任者である番浩志は、原版からの機能継承を確認し、動作チェックを行いながら開発を進めています。ソフトウェアと公開版の責任は作者が担います。

02 クイックスタート

レチノトピー刺激を実験室のPCで準備し、スキャナの開始信号に合わせて呈示し、取得後のタイミングを確認するための手順書です。

まずはこの順序で

1 環境を確認	2 装置に合わせる	3 取得前に準備	4 実行・記録確認
同梱Pythonでlistと短時間デモを実行	画面寸法・距離・refresh・眼割当を設定	画像バンクと動画キャッシュを用意	開始信号を待ち、終了後にsummaryと境界誤差を確認

PowerShellでの共通準備

以下は、プロジェクトをE:¥PyRetinotopy¥PyRetinotopyへ配置した場合のPowerShell例です。設置場所とPsychoPyのPythonパスを環境に合わせて変更してください。pyrtは、このターミナル内だけで使う短縮関数です。

```
Set-Location 'E:\PyRetinotopy\PyRetinotopy'
$retinoPython = 'C:\Program Files\PsychoPy\python.exe'
function pyrt {
  & $retinoPython -m pyretinotopy @args
}
pyrt list
pyrt validate --config examples\ccwf.json
pyrt run --config examples\smoke.json
```

smoke.jsonは640×480の短い自動開始デモです。同じ取得の再実行は前のフォルダ全体を連番退避します。別の撮像は--run Nで区別します。

本書の案内

3：刺激 4：開始位置 5：被験者設定 6：両眼表示 7：画像 8：取得 9：ラン・再試行 10：原版設定 11：時間 12：結果 13：対処 14：引用 15：謝辞

03 刺激モードを選ぶ

基本モード14種類と接尾辞5種類の組合せで70モードを指定できます。刺激形状と表示方式は独立した設定です。

基本モード	内容
ccw / cw	反時計回り / 時計回りに回転する楔形
exp / cont	拡大 / 縮小する環状刺激
bar	複数方向へ走査するバー（pRF用）
ccwexp / ccwcont	反時計回りの楔形 + 拡大 / 縮小環
cwexp / cwcont	時計回りの楔形 + 拡大 / 縮小環
multifocal	極角×離心度の領域をm系列または指定デザインで制御
meridian	水平 / 垂直子午線の対向する楔形
lgn	左右半視野のLGN localizer
hrf	刺激と休止を繰り返すHRF推定用刺激
localizer	指定離心度帯とその補領域

接尾辞	キャリア / 課題	例
なし	カラーチェッカー / 刺激内の輝度低下検出	ccw
f	カラーチェッカー / 注視点課題	ccwf
d	ランダムドットステレオグラム / 奥行き変化検出	ccwd
df	ランダムドットステレオグラム / 注視点課題	ccwdf
i	物体画像 / 注視点課題	ccwi

時間の設定項目

楔形・環・HRFはcycle_sとrepeats、バーはcycle_s×方向数×repeats、multifocalはtrial_s×系列長、子午線・LGN・localizerはblock_s×2×repeatsが刺激部分の長さです。前後にinitial_fixation_sとfinal_fixation_sを加えます。

複合刺激ではpolar_*とeccentricity_*の周期を独立に指定します。両者の総時間を一致させてください。rest_s等は各周期・ブロック・試行の内部に含まれます。設定の総時間はvalidateのduration_sで確認します。

独立JSON方式ではプロトコルごとにinit --modeで設定を作成します。既存JSONへの--mode指定は名前だけの変更で、他の設定を刺激の既定値へ戻しません。継承方式では--subjectと--modeを指定します。

04 開始位置とPTB3との対応

v0.3.0では原版の画像座標と生成画像の逆順を反映し、全70モードの開始位置・走査位置を照合しました。刺激の前には既定で4秒の注視があります。

刺激	既定の開始位置・規則
cw / ccw	cwは真上（中心90度）、ccwは真上から反時計回りへ12度（中心102度）。ccwは全30画像を逆順にする原版の先頭位置です。
exp / cont	expは中心側から拡大。contは10秒休止の後、外周側から縮小。単独の環は25位置です。
複合 / bar	複合環は16位置・2秒刻み・公称幅1度。バーの最初の走査は右端から左へ。
multifocal / LGN	multifocalは時計回りの領域配置。LGNは左側から始まり、水平反転した右側と交互。
meridian / HRF / localizer	水平子午線／全周／指定離心度帯から、それぞれ開始します。

開始角と互換設定

stimulus.spatial_conventionの既定値は"ptb3"です。設定のstart_angle_degとdual_ring_start_angle_degは原版の画像座標（右0度・時計回りが正）です。内部成分と出力窓は視野座標（上向きが正）へ変換します。

multifocal_indexingの既定値"corrected"は36領域の重複を解消します。原版のゼロ位相時のID重複まで再現する場合は"ptb3"とします（phase 0、幅360度のみ）。採用した規則を保存設定とともに記録してください。

v0.2.2以前のPython呈示を再現するにはspatial_conventionを"legacy-python"とします。旧config.jsonに項目がなく、隣のsummary.jsonが0.2.2以前を記録していれば自動認識します。通常の設定JSONには新しい既定値を適用します。

原版の式を別実装した連続座標マスクで全位置を照合済みです。MATLABを実行した画素一致の試験ではなく、境界の丸め、課題列や乱数キャリアの同一性は主張しません。docs/PTB3_PARITY.ja.mdに式・比較図・限界をまとめています。

05 環境と参加者設定を用意する

確認済み環境とインストール

確認済みはWindows、PsychoPy 2024.2.4、Python 3.10.11です。基本依存はNumPyとPillow。呈示にはPsychoPyを使います。MATLABの導入は不要です。通常Python環境では、PsychoPyが動作する環境でpip install -e .を実行できます。

Windowsで撮像用の全画面呈示を行う場合はGLFWを使います。このPCでは下記のプロジェクト内導入を済ませています。別PCでは作業ディレクトリから一度だけ実行してください。

```
& $retinoPython -m pip install --target optional-libs --no-deps `
  psychopy-glfw==0.0.2 glfw==2.8.0
pyrt init-subject HB --mode ccwf
```

原版の設定フォルダに対応するsubjects/_DEFAULT_/を用意しています。common.json、retinotopy_display.json、刺激別c_*.json、任意のモード名.jsonを順に読み、同じ順序のsubjects/HB/の値で上書きします。被験者側は変更したい項目だけ記載できます。

JSONを装置と実験に合わせて編集する

項目	設定する内容
subject / run / seed	参加者ID、取得番号、乱数seed。各ランを識別できるように保存。
display	mode、size_px、screens、width_cm、distance_cm、refresh_hz、backend、full_screen。
timing	前後注視、周期・繰返し・休止、開始方法、トリガーと反応キー。
stimulus	半径・楔形幅・バー方向・色・視差・ドット・物体画像等。
task	注視点type/size/color、fixation_frequency、duration_s、schedule、反応window。

視角と画面の校正

width_cmにはsize_px全体に対応する物理幅、distance_cmには眼から表示面までの距離を入れます。左右分割では全画面幅を入れます。視角換算はsize_px[0] / width_cm × distance_cm × tan(1度)のpixels/degreeです。各眼の領域に刺激が入らない場合はエラーになります。

実際のframebuffer画素数がsize_pxと異なる場合や、安定したrefreshを計測できない場合は開始しません。OSの表示倍率と解像度、表示装置の設定を確認してください。設定例の物理寸法は校正済みの値ではありません。

```
pyrt validate --subject HB --mode ccwf
pyrt preview --subject HB --mode ccwf --output preview.png
```

継承は--subjectで利用します。--config FILEは独立したJSON指定で、別の既定ファイルを探索しません。保存済み取得条件はこの方法で再実行できます。詳細はsubjects/README.mdを参照してください。

validateは設定・スケジュールの検査です。実機校正、左右眼の分離、光学時刻や性能の確認は別に行います。

06 単眼・両眼の表示を合わせる

display.mode	配置と確認事項
mono	1ウィンドウに片眼像を呈示。
parallel / cross	左右分割。parallelは左眼像が左側、crossは逆。
dualparallel / dualcross	1枚の横長framebufferを左右分割。OS側の画面構成を確認。
dual	異なるscreen IDの2ウィンドウ。両眼のswap時刻差を記録。
topbottom / bottomtop	上下分割。topbottomは左眼像が上側、bottomtopは逆。
redgreen / greenred	赤と緑のchannelに輝度投影。後者は眼の割当を逆転。
redblue / bluered	赤と青緑側channelに輝度投影。原版のgainを使用。
interleavedline / interleavedcolumn	偶数行 / 列に左眼像、奇数側に右眼像。機器の原点を確認。
shutter	quad-buffer stereo。対応GPU・ドライバ・表示機器が必要。

位置と反転

eye_offsets_pxは各眼の[x, y]で、右と上が正です。mirror_x / mirror_yで各眼の全シーンを水平・垂直反転できます。反転は注視点・マーカーと位置補正も含めて作用するため、実際に片眼ずつ確認してください。画像を片眼領域へ押し縮める処理は行いません。

立体視を使う前の確認順序

1. 片眼ずつ遮蔽し、想定する刺激がその眼に届くことを確認します。2. 注視点と視野端を使って位置・大きさ・鏡像を調整します。3. RDSの負の視差が手前に知覚されることを確認します。4. 眼鏡を通した漏れと輝度を測定します。

dualの左右swapは順次で、ハードウェア同期を保証しません。frames.tsvのeye_skew_sを確認し、必要に応じて両眼を光学計測します。shutterはquad bufferが使えない場合にエラーで停止します。

表示設定の推奨開始点

full_screen=true、backend="glfw"、high_priority=true、realtime_priority=falseから装置評価を始めます。最大優先度の方が速いとは限りません。gammaは各channelの逆べき乗補正です。任意のハードウェアgamma LUTの自動移行は行いません。

PROPixx / Datapixxの専用制御は未実装です。通常のquad bufferや分割表示と同等と仮定しないでください。

07 物体画像を取得前に準備する

MATLABから独立した画像バンク

物体刺激には、assets/object-images/images.npyとmanifest.jsonを用意します。標準バンクは7,493枚のuint8画像で、配列順は画像×縦×横×RGBAです。変換時には全画素とalphaの一致を検査し、manifest.jsonに形状と画素digestを保存します。呈示にはMATファイルやSciPy/h5pyを使用しません。

別PCにはimages.npyとmanifest.jsonを一緒に配置してください。大容量バンクはソースコードのGit管理から除外しています。通常のPNG等を置いたディレクトリもobject_sourceとして指定できます。

旧画像バンクから一度だけ再変換する場合

```
pyrt convert-images `
  ..\Retinotopy\object_images\object_image_database.mat `
  --output assets\object-images
```

この一回の変換にはSciPy/h5pyが必要です。必要な場合だけpip install -e .[legacy]で追加します。変換元は読み取り専用です。既存images.npyへの上書きは拒否します。

設定ファイルから見たパス

object_sourceとobject_cache_dirの相対パスはJSONの置き場所が基準です。たとえばsubjects/HB/ccwi.jsonからプロジェクト直下を指す場合は以下です（stimulus部分の抜粋）。

```
"object_source": "../../assets/object-images/manifest.json",
"object_cache_dir": "../../cache/object-movies",
"object_count": 120,
"object_update_s": 0.25,
"object_texture_slots": 4
```

画像の合成を全て終えてからトリガーを待つ

```
pyrt prepare-objects --config examples\object-images-fullscreen.json
pyrt run --config examples\object-images-fullscreen.json
```

全モザイクの変換・回転・ノイズ・gamma・anaglyphを取得前に準備します。設定とseed等が同じなら再利用し、条件が変わると別キャッシュになります。runも未準備の場合は画面初期化前に自動生成します。

360秒、1031×1031の左右像、250 ms更新では約9.2 GBの動画キャッシュを使います。元バンク約1.96 GBと合わせて空き容量を確保してください。取得中は既成画像の先読みとGPU転送を行い、準備不足時はログを残して停止します。

08 当日の取得を実行する

1 設定と準備を確認する

参加者ID、run番号、seed、画面設定、周期と総時間、トリガー・反応キーを照合します。物体刺激はprepare-objectsを完了しておきます。実験中に更新・通知・他のGPUアプリが動かない状態を作り、同じ設定で事前に通常ラン長の試験を行います。

```
pyrt validate --subject HB --mode ccwf
pyrt run --subject HB --mode ccwf
```

2 開始信号を入力する

timing.start	開始操作
trigger（既定）	trigger_keysに指定したキー。既定はtまたは5。
key	SpaceまたはEnter。
mouse	左クリック。
immediate	画面と準備の完了後、自動開始。デモ・性能確認用。

skip_triggers=0は最初のパルス、4なら最初の4パルスを無視し5個目で開始します。開始前から注視点が表示されます。開始トリガーを受けた後の第1実測flipをラン0秒とし、前置注視もラン時間に含みます。

USB等でスキャナパルスをキーへ変換して受けます。平行ポートの電圧を直接読む機能は含みません。トリガー時刻と第1flipの差は保存されたevents.tsvで確認できます。

3 課題反応と中止

既定の反応キーはr / g、中止はEscape／Qです。課題のRTは実際の課題呈示flipを起点に計算します。中止時は終端blankを表示し、到達した範囲のログを保存します。未到達境界はn/aです。開始待ち中にもEscape／Qで中止できます。

4 結果を残し、次のランを用意する

結果はsubjects/HB/results/261106/ccwf_run-01/のように被験者別・日付別に保存します。日付YYMMDDはラン開始処理時の実験PCのローカル日付です。別の撮像は--run Nで区別し、同じ番号の再実行では前の結果を連番退避します。結果だけがあるIDへinit-subjectで設定を追加することもできます。

継承設定の--subjects-dirや、IDフォルダ内に置いた明示JSONの設定ルートを保存先にも使います。例のJSONはプロジェクトのsubjectsへ保存します。--output-root ROOTを明示した場合だけ、別保存先ROOT/sub-ID/モード_run番号/を使います。既存dataは自動移動しません。

```
pyrt session examples\session.json
```

session.jsonには設定ファイルとrun番号を列挙します。各ランで開始を待つには各設定のstartをtriggerにします。中止すると残りのランも止まります。

09 ラン番号と同じランのやり直し

同じ刺激を複数回撮像する場合は--runでラン番号を指定します。引数はJSON内のrunより優先され、設定・summary・保存フォルダへ反映されます。

```
pyrt run --subject HB --mode ccw --run 1
pyrt run --subject HB --mode ccw --run 2
# Repeat run 1 after motion or a response problem
pyrt run --subject HB --mode ccw --run 1
```

同じ日・ID・mode・runの再実行では、前の取得フォルダ全体を連番退避し、最新の試行を元のフォルダ名へ保存します。異なるrun番号と既存backupは変更しません。

```
subjects/HB/results/261106/
  ccw_run-01/          # latest attempt
  ccw_run-01_old-01/   # first previous attempt
  ccw_run-01_old-02/   # next previous attempt
  ccw_run-02/          # a separate scan
```

設定・選択	動作
--if-exists backup	既定。同じ番号を連番退避して再実行。
--if-exists error	既存結果があれば変更せず取消。
--no-confirm	実行前の設定確認を省略。スキャナ待ちは維持。

既定のoperator_confirm=trueでは、退避・準備前にID・mode・runを確認します。取消なら既存データを動かしません。中止・失敗の試行も保持するので、採用データはsummary.jsonのstatusと各ログで確認してください。

acquisition.jsonに一意IDと直前結果の退避先を保存します。OSロックを最終保存まで保持し、実行中の同一ランの退避を防ぎます。.lock管理ファイルは正常終了後も残り、プロセス終了時にはロックだけが解放されます。

Coderとsessionも共通処理です。sessionの各項目にrunを設定し、異なる撮像を区別します。同一session内の同じ保存先の重複はエラーです。

10 原版の注視点・課題・表示設定

PTB3	fixation_type	形状
1	circle	円形。
2	cross	枠と目盛状の十字。単純な塗りつぶし四角とは異なります。
3	concentric	中央点と切れ目のある外周。

task.fixation_radius_pxとfixation_colorを指定します。課題の暗色は既定で[64,64,64]/255です。squareはPython追加選択として残します。

```
"task": {
  "fixation_type": "concentric",
  "fixation_radius_px": 12,
  "schedule": "ptb3",
  "fixation_frequency": 4,
  "duration_s": 0.3
}
```

ptb3方式の注視課題は原版のfixtaskfreqと期間slotを使い、連続する課題を避けます。前後注視・休止中も対象です。刺激内課題は位置／色／視差の半期間を使います。interval方式なら従来のinterval_sとduration_sを使います。

displayの項目	用途
background_type = grid	周辺の左右共通固視ガイド。patchのサイズ・数・2色と中央空白域。
marker_timing = half	位置保持の半分の点灯。fixedならmarker_duration_s。
gamma_lut / gamma_lut_right	native JSONのN×3 RGB表。rightはdualの第2物理画面。
use_measured_refresh = true	原版force_frame_rate=0相当。実測refreshを保存。

色列・RDSは最初の活動呈示から始まり休止で停止します。RDSは対向checker面の視差を段階循環します。全設定の対応、旧Python版の再現、機器依存の未対応はdocs/PTB3_FEATURES.ja.mdとPTB3_PARAMETER_MAP.ja.mdに記録しています。

11 ブロック境界を累積時刻で守る

全刺激でラン開始の第1実測flip時刻t0を固定します。ブロックの実測終了を次の起点へ置き換えません。前置注視4秒・周期60秒なら、境界の目標は次の累積時刻です。

周期	ラン開始から固定した目標
1	t0 + 4秒
2	t0 + 64秒
3	t0 + 124秒
4	t0 + 184秒
5	t0 + 244秒
6	t0 + 304秒

上表は前置注視4秒・周期60秒の場合の累積目標時刻です。実際の切替時刻は各取得のblock_timing.tsvに保存されます。planned_onset_s、actual_onset_s、error_sを確認し、後半ほど誤差が増えていないかを点検してください。

各flipで選ぶ刺激

PsychoPyが予測する次のflipの累積時刻から名目refreshのスロットを選択します。表示が少し速い場合は状態を保持し、遅い場合は現在時刻の状態へ進めます。毎flipで名目番号を必ず1増やす制約を設けないことで、refreshの微差による長時間のズレも避けます。

楔形・環・HRFは周期、バーは各走査、multifocalは各試行、子午線・LGN・localizerは各ブロックが同じt0を使います。楔形＋環は2種類の独立周期を同じt0から計算します。マスク位置の切替と課題もラン時間で制御します。

ブロック境界の記録を読む

列	意味
planned_onset_s	ラン開始からの連続時間の目標。
scheduled_onset_s	目標を名目refreshへ丸めた予定時刻。
actual_onset_s	該当状態を表示した実測flip時刻。
error_s / frame	実測 - 目標（秒） / 実flip通し番号。

リフレッシュによる離散化誤差とOS・GPUのジッタは残ります。遅れた瞬間そのものを取り消すことはできません。PsychoPyの同期時刻と、眼に届く光の時刻は区別し、撮像装置で光学的にも確認してください。

12 取得結果を点検して解析へ渡す

保存ファイル	主な確認内容
acquisition.json	一意ID、run番号、直前結果の退避先。
config.json	その取得に使用した設定・seed。画像とcacheの絶対パス。
summary.json	completed / aborted / failed、エラー、flip統計、境界最大誤差、環境。
frames.tsv	予定時刻、実測flip、間隔、状態、carrier更新、CPU時間、左右swap差。
block_timing.tsv	各周期・走査・試行・ブロックの予定 / 実測 / 誤差。
events.tsv / events.json	刺激、blank、トリガー、反応、課題、終了の時刻と説明。
targets.tsv / responses.tsv	課題の予定・実測開始、反応分類、RT。
multifocal_design.npy	multifocalで実際に使用した領域×時点のデザイン。

取得後の確認順序

まずstatusとerrorを確認し、timing.long_intervals、estimated_missed_frames、max_interval_sを確認します。長い間隔は名目refreshの1.5倍超です。60 Hzなら25 ms超となります。推定欠落はflip間隔から求める指標で、光学測定したフレーム数ではありません。

次にblock_timing.tsvのerror_sを時系列に確認します。最大値に加え、後半ほど増える傾向がないかを確認します。frames.tsvのupdate_cpu_s、draw_cpu_s、upload_cpu_sで負荷の場所を調べられます。無人試験の課題missは想定内です。

解析用の刺激窓を出力する

```
pyrt export --config examples\barf.json `
  --output bar-windows.npz --tr 2 --resolution 128
$resultDay = Get-Date -Format yyMMdd
$runDir = "subjects\HB\results\$resultDay\ccwf_run-01"
pyrt export --config "$runDir\config.json" `
  --frames "$runDir\frames.tsv" `
  --output actual-windows.npz --prt actual.prt --tr 2
```

1つ目は計画上の窓、2つ目は実flipに基づく窓です。実flipの窓は次のflipまで直前の状態が保持されたものとしてTR内露出を積分します。NPZの軸は時間×y×x、上側の行が視野上方です。MAT互換出力は任意で、y×x×時間です。

窓は幾何学的な刺激領域を表します。注視点、ノイズ、物体そのもの、視差、装置の光学遅延は含みません。ログ一式だけで完全なBIDSデータセットにはなりません。

--prtで同じ実測区間のBrainVoyager PRTを出力します。export-olts --output DIRは原版6色表、preview --all-positions --output DIRは全位置とmanifestを保存します。

13 困ったときと動作確認

症状	確認・対応
長いflip間隔が出る	全画面・GLFW・native解像度を確認。他のGPU処理を止め、同じ通常ラン長で再測定。最大優先度を常用しない。
Object epoch / prefetchのエラー	事前動画とSSDの転送速度、空き容量を確認。画像枚数・サイズ・更新周期を見直し、別runで再測定。
画像バンクが見つからない	JSON基準の相対パスとimages.npyの存在を確認。MATは一度convert-imagesで変換。
framebuffer / refreshのエラー	size_px、画面番号、全画面の解像度、OSの表示倍率・refreshを装置と一致させる。
取得ディレクトリが存在する	既定は連番backup。error指定時は取消。実行中なら終了を待つ。
左右眼・視差が逆	片眼ずつ遮蔽しmodeとmirror設定、位置補正、眼鏡側の眼割当を確認。

取得前の動作確認

刺激の設定と位置、累積時刻、左右眼の割当、課題、結果保存とバックアップ、PRT／OLT出力を、実験で使用する条件に合わせて確認してください。ソフトウェアの自動テストはpython -m unittest discover -s tests -vで実行できます。

刺激呈示にはハードウェアOpenGLが必要です。リモートデスクトップでは利用できない場合があるため、GPUが利用できるローカルデスクトップを使用してください。時間精度と通常ラン長での性能は、実際のスキャナ・表示装置・反応装置を組み合わせて確認します。

参照する文書・ライセンス

README.md：全体説明・研究背景・参考文献。docs/USAGE.ja.md：実行手順。docs/PRESENTATION_TIMING.ja.md：時刻の設計。docs/PTB3_FEATURES.ja.mdとdocs/PTB3_PARAMETER_MAP.ja.md：原版との機能・設定の対応。docs/PTB3_PARITY.ja.md：開始位置・走査順と互換性。

ソースは原版のBSD条件を保持しています（LICENSE.txt）。原版・画像・依存ソフトの謝辞はNOTICE.mdを参照してください。画像の変換は、その出典や利用条件を変更しません。研究では原版Retinotopy、PyRetinotopyの使用版、PsychoPy、使用した画像データベースを適切に引用してください。

原版：github.com/hiroshiban/Retinotopy
PsychoPy：psychopy.org
GLFW plugin：github.com/psychopy/psychopy-glfw

14 引用・参考文献・ライセンス

ソフトウェアの引用

研究ではPyRetinotopy（番浩志）の使用版またはcommitと、原版Retinotopy toolbox（番浩志）を記載してください。使用したPsychoPyの版に対応する引用も行ってください。原版URL：https://github.com/hiroshiban/Retinotopy

物体画像を使用する場合

画像の変換は出典や利用条件を変更しません。該当するデータベースの論文を引用してください。主なデータベース論文を以下に示します。

1. Konkle, T., & Caramazza, A. (2013). Tripartite Organization of the Ventral Stream by Animacy and Object Size. Journal of Neuroscience, 33(25), 10235-10242.

2. Konkle, T., & Oliva, A. (2012). A real-world size organization of object responses in occipito-temporal cortex. Neuron, 74(6), 1114-1124.

3. Brady, T. F., Konkle, T., Alvarez, G. A., & Oliva, A. (2008). Visual long-term memory has a massive storage capacity for object details. PNAS, 105(38), 14325-14329.

4. Konkle, T., Brady, T. F., Alvarez, G. A., & Oliva, A. (2010). Conceptual distinctiveness supports detailed visual long-term memory. Journal of Experimental Psychology: General, 139(3), 558-578.

5. Konkle, T., & Oliva, A. (2012). A Familiar Size Stroop Effect: Real-world size is an automatic property of object representation. Journal of Experimental Psychology: Human Perception and Performance, 38, 561-569.

原版の研究背景

Yamamoto, H., Ban, H., et al. (2009). Large- and Small-Scale Functional Organization of Visual Field Representation in the Human Visual Cortex. In Visual Cortex: New Research, 195-226.

Ban, H., Yamamoto, H., et al. (2013). Topographic processing of an occluded object and the effects of spatiotemporal contexts in human early visual areas. Journal of Neuroscience, 33(43), 16992-17002. DOI: 10.1523/JNEUROSCI.1455-12.2013

Chou, I. W. Y., Ban, H., & Chang, D. H. F. (2021). Modulations of depth responses in the human brain by object context: Does biological relevance matter? eNeuro. DOI: 10.1523/ENEURO.0039-21.2021

配布条件

ソースには原版のBSD条件、著作権表示、免責条項を保持しています。詳細はLICENSE.txtとNOTICE.mdを参照してください。画像データ・PsychoPy・GLFWその他の依存物には、それぞれの利用条件が適用されます。

15 謝辞

PyRetinotopyと前身のRetinotopy刺激の開発経緯

PyRetinotopyとその前身であるRetinotopyは、私が英国のUniversity of Birminghamに在籍し、Andrew Welchman博士およびZoe Kourtzi博士の指導の下で開発・使用していたレチノトピー刺激に、さまざまな改変を加えた一連のプログラムをもとにしています。ここに、お二人のご支援に深く感謝いたします。

これらの刺激の基礎となったのは、京都大学のHiroki Yamamoto博士とYoshimichi Ejima博士が、C/C++およびVisualization Toolkit（VTK、Kitware Inc.）ライブラリを用いて開発されたコードです。これらのコードはSolaris/IRIXのワークステーションで動作していました。私はお二人から、視覚刺激の作り方について多くを学びました。

これらの元の刺激をMATLABとPsychtoolboxで全面的に改変・更新し、長年にわたって新たな機能や刺激の種類を追加してきたものが、原版Retinotopyです。

最近の心理学・神経科学分野でのPythonの利用の広がりを受けて、MATLAB Psychtoolbox 3で作成した原版Retinotopyの刺激を、PsychoPyで動作するようにPythonで書き換えたものが、本パッケージPyRetinotopyです。

PsychtoolboxとPsychoPy

原版を支えたPsychtoolboxの開発者・貢献者、David Brainard、Denis Pelli、Allen Ingling、Mario Kleiner、Richard F. Murray、Diederick C. Niehorster、Tobias Wolfの各氏に感謝します。Python版の呈示と反応記録を支えるPsychoPyの開発者と貢献者にも深く感謝します。

任意のGLFW経路にはMatthew Cutone / Open Science Toolsによる公式psychopy-glfwプラグインを使用します。NumPy、Pillow、ならびに変換・文書作成等を支えるオープンソースソフトウェアの開発者にも感謝します。

画像データベースとm系列

物体画像データベースの開発・公開に貢献されたTalia Konkle氏の研究グループと共同研究者の皆様に感謝します。画像を用いた研究では前ページの該当論文を引用してください。

m系列生成コードを開発されたGiedrius T. Buracas氏と、それを拡張されたThomas Liu氏に感謝します。このコードには、私自身もその後さらに修正を加えています。

Python/PsychoPy改訂への生成AIのサポート

PyRetinotopyの開発では、MATLAB Psychtoolbox 3版RetinotopyからPython/PsychoPyへの書換えに、OpenAIの生成AI（Codexを含む）のサポートを受けました。コードの調査、設計、実装、テスト、文書作成に利用しています。

本ソフトウェアが、視覚科学・神経科学の研究と学習に役立つことを願います。

PyRetinotopyの作者・責任者である番浩志は、原版からの機能継承を確認し、動作チェックを行いながら開発を進めています。ソフトウェアと公開版の責任は作者が担います。